

Contents

GPSE Changelog — Per-Commit Update History	1
2026-07-18	2
ebea4e6 — feat(cli): make gpse 42 easter egg more theatrical	2
2026-07-20	3
baf898c — feat: harden train and prediction pipeline	3
20553c1 — fix(train): improve SVR convergence and log headers	4
4fa75b0 — fix(models): tighten kernelridge_reg search space to	4
c0c791c — fix(train): scope CatBoost outputs to results directory	5
6835627 — docs(wiki): add wiki framework for CLI subcommands and	5
1c51409 — fix(utils): skip correlation on constant inputs to	6
3fe1c39 — style(log): collapse fold results into a single line	7
cc4820e — style(log): tag fold/repeat log lines with model name	7
409b152 — fix(batch): stop remaining tasks on Ctrl+C interrupt	8
bcacd92 — feat(train): recycle leftover thread budget into	8
5c811cf — fix(train): drop invalid n_jobs for HistGBR; colorize	9
d7f863e — fix(log): use colorized stderr sink for default console	9
e0d0d6f — Revert “fix(log): use colorized stderr sink for default	10
6298af4 — chore(log): reduce noisy separators and fix wrapped log	10
1ed2b1b — feat(convert): add Azodi 2019 conversion mode (-101	11
a3712aa — chore(log): shorten long paths in logs and fix pandas	12
ceafb6c — feat(train): save final hold-out predictions from	13
c4b0ce0 — feat(batch): add gpse batch subcommand for multi-trait	14
2026-07-21	14
e19517d — feat(batch): unify batch logging with train style and	14
1bfd689 — docs(readme): add PyPI install instructions and version	15
08a226e — fix(train): work around scipy-OpenBLAS segfault in	15
b72ee41 — feat(batch): show single-line command and derived config	16
74c9b36 — feat: add gpse tools split subcommand for train/test	17
98411c8 — docs: move train_pipeline.md to docs/	18
597f1d7 — feat(batch): merge per-trait summary tables across	18
a35b8cf — chore(validation): run all 15 regression models in	19
445bab6 — chore(release): bump to 0.0.3a1 with validation suite	19
2026-07-22	20
d3c9e5d — perf: optimize hot-path bottlenecks in training and	20
386aa5a — feat(topsis): expand criteria config, add	21
32d107a — feat(models): YAML-driven model registry with	22
d45d03a — gpse add framework	23
aa76e5e — fix(registry): make inline search-space param funcs	23
2026-08-20	24
e7eff28 — fix(convert): allow -bfile as alternative genotype	24
002ad72 — Merge branch ‘main’ of github.com:xsx123123/GPSE into	25
b99e741 — Release GPSE 0.0.4a	25

GPSE Changelog — Per-Commit Update History

Project: GPSE — Genomic Prediction with Stacking Ensemble for horticultural crops **Repository:** <https://github.com/xsx123123/GPSE> **Coverage:** 2026-07-18 → 2026-08-20 (34 days, 36 commits, versions 0.0.2 → 0.0.4a) **Generated:** 2026-09-07, from `git log / git show` of each commit

Each entry documents one commit: a short **Description**, the key **Code** (trimmed diff or snippet), and the concrete **Changes**. Entries are grouped by date in chronological order.

2026-07-18

ebea4e6 — feat(cli): make gpse 42 easter egg more theatrical

Type: feat | **Date:** 2026-07-18

Description: Reworks the hidden `gpse 42` Hitchhiker’s-Guide easter egg into a multi-act terminal animation: a spinner “thinking” phase, a line-by-line FIGlet-style ASCII “42” reveal, a centered typewriter answer, and a randomly chosen afterglow tagline mixing Hitchhiker and genomics jokes. All rendering stays pure-ASCII-safe so it works in any terminal, with a plain-print fallback when `rich` is unavailable.

Code:

```
_ASCII_42 = r"""  
      |   |   |       |     \    |\n  
      |   |   |       )    | |\n  
      |___|__|       /   /  /|\n  
            |   |       //  /_\n  
            |_|        |_____|  
  
"""  
  
_THINKING_LINES = (  
    "Consulting Deep Thought...",  
    "Enumerating life, the universe, and everything...",  
    "Sampling random_seed=42 for reproducibility...",  
    ...  
)  
  
_AFTERGLOW = (  
    "Don't Panic – and always carry a towel.",  
    "Mostly harmless. Highly heritable.",  
    "So long, and thanks for all the phenotypes.",  
    ...  
)  
  
def print_easter_egg(show_question: bool = False) -> None:  
    """Display a theatrical Hitchhiker-meets-genomics easter egg."""  
    # Act I – spinner Progress over _THINKING_LINES  
    # Act II – Live panel: ASCII 42 revealed line-by-line, then  
    #             typewriter-styled two-line answer + afterglow footer
```

Changes: - gpse/utils/cli_display.py: print_easter_egg() rewritten into Act I (spinner “computing” phase via rich.progress.Progress) and Act II (rich.live.Live panel revealing ASCII art then typing the answer). - Answer split into two centered lines (_ANSWER_LINE_1/_ANSWER_LINE_2) so the panel never wraps awkwardly; per-line styled segment painting added. - New rotating _AFTERGLOW taglines (10 Hitchhiker/genomics lines) picked with random.choice, shown as a dim italic footer once typing completes. - Panel subtitle changed to Don't Panic · Share and Enjoy; non-rich fallback prints the ASCII art, first 3 thinking lines, and one afterglow line. - No functional/CLI surface change; cosmetic-only commit touching a single file (171 insertions, 44 deletions).

2026-07-20

baf898c — feat: harden train and prediction pipeline

Type: feat | **Date:** 2026-07-20

Description: Large overhaul that makes the training and prediction pipeline reproducible and portable. It introduces canonical SNP identifiers, a `gpse predict` implementation with feature alignment against a saved manifest, fixed hold-out splits with persisted manifests and reports, leak-free fold-local feature selection/imputation, portable result bundles, and a major refactor of the stacking ensemble module, plus extensive docs and tests (5,600 insertions across 36 files).

Code:

```
+# gpse/utils/snp_ids.py - canonical SNP IDs shared by convert/train/predict
+def canonical_snp_id(chromosome, position, reference=None) -> str:
+    """Build ``chr<chrom>_<chromStart>_<chromEnd>`` from a 1-based VCF position
+    using UCSC-style zero-based, half-open coordinates (POS-1_POS)."""
+    start = int(position) - 1
+    end = start + max(len(reference or ""), 1)
+    return f"{normalize_chromosome(chromosome)}_{start}_{end}"

+# gpse/utils/feature_manifest.py - ordered feature list saved next to models
+FEATURE_MANIFEST_NAME = "feature_manifest.json"
+def write_feature_manifest(directory, feature_names, *, source_file=None): ...
+def find_feature_manifest(artifact_path): ... # search artifact parents

+# gpse/predict/cli.py - predict CLI is now real (was a stub)
+parser.add_argument("--model", required=True, help="Model artifact or GPSE results directory.")
+parser.add_argument("--vcf-file", "--geno-file", dest="genotype_file", required=True)
+parser.add_argument("--out", required=True, help="Prediction CSV output file.")
+parser.add_argument("--report", help="Alignment report JSON ...")
+parser.add_argument("--missing-value", type=float, default=3.0)
+parser.add_argument("--min-feature-coverage", "--min_feature_coverage", type=float, default=0.0)

+# gpse/train/_pipeline.py - fixed hold-out split + persisted reports
+train_indices, fixed_test_indices, split_metadata = create_holdout_indices(
+    y, self.test_size, self.random_seed, self.task_type, X=X,
+    split_strategy=self.split_strategy, ..., return_metadata=True)
+_save_split_manifest(...) # train_ids.txt / test_ids.txt / split_manifest.json
+_write_holdout_reports(...) # reports/model_comparison_holdout.csv, holdout_metrics.{csv,json}
```

Changes: - New `gpse/utils/snp_ids.py`: canonical `chr<chrom>_<start>_<end>` SNP IDs (0-based half-open), PLINK MAP/BIM readers, and duplicate-ID validation shared across `convert/train/predict`; covered by new tests/`test_snp_id_alignment.py`.

- New `gpse/utils/feature_manifest.py`: writes/reads

`feature_manifest.json` (schema_version, feature_id_format, ordered unique feature names) so trained models are portable. - New `gpse/predict/core.py` + rewired `gpse/predict/cli.py`: `predict` resolves model artifacts (deployment ensemble dir, stacking pkl, representative model), loads VCF (via `cyvcf2`, additive 0/1/2 coding, missing=3) or converted matrices, aligns features to the manifest, fills missing SNPs, enforces `--min-feature-coverage`, and emits an alignment report JSON. - New `gpse/train/_feature_selection.py`: leak-free, fold-local feature selection (none/variance/univariate/mutual_info/tree_importance/lasso/elasticnet/pca/svd) and genotype imputation configs fitted inside each CV fold. - `gpse/train/_pipeline.py` + `_repeat_training.py`: `use_same_test_set` now builds one fixed hold-out via `create_holdout_indices` (stratified for classification), persists `split_manifest.json` and `train_ids.txt/test_ids.txt`, and writes final hold-out reports under `reports/`. - New `gpse/train/_results.py`: read-only result bundles (leaderboard, markdown/HTML report, schema-versioned

JSON) written after training; gpse/train/stacking.py slimmed from 688 changed lines into supporting modules (_ensemble.py, _model_pipeline.py, _cv_manager.py, _fold_training.py). - Docs added: train_pipeline.md, docs/train_predict_audit_report.md, docs/todo_v3.md, change.md, README updates; new tests/test_train_pipeline_optimization.py (631 lines).

20553c1 — fix(train): improve SVR convergence and log headers

Type: fix | **Date:** 2026-07-20

Description: Fixes LinearSVR frequently hitting its default iteration limit by giving it an explicit `max_iter=10_000` budget in both the Optuna search space and the default parameters. Also removes stray leading newlines in log separator lines so log headers align cleanly.

Code:

```

+## gpse/models/regression_model_optimizer.py
+LINEAR_SVR_MAX_ITER = 10_000
+
+    def _svr_reg_params(self, trial):
+        params = {
+            'loss': trial.suggest_categorical('loss', ['epsilon_insensitive',
+'squared_epsilon_insensitive']),
+            'C': trial.suggest_float('C', 1e-5, 1e2, log=True),
+            'tol': trial.suggest_float('tol', 1e-8, 1e2, log=True),
+            'max_iter': LINEAR_SVR_MAX_ITER,
+            'random_state': self.random_seed
+        }
+
+    # default params for 'svr_reg' also gain 'max_iter': LINEAR_SVR_MAX_ITER
+
+-## gpse/train/pipeline.py & _repeat_training.py
+-main_logger.info(f"\n{'=' * 70}")
+main_logger.info(f"{'=' * 70}")

```

Changes: - gpse/models/regression_model_optimizer.py: new module constant `LINEAR_SVR_MAX_ITER = 10_000`; `max_iter` added to both the `svr_reg` Optuna param function and its default params. - gpse/train/_pipeline.py, gpse/train/_repeat_training.py: separator log lines no longer start with `\n` (4 call sites), cleaning up log header formatting. - tests/test_train_pipeline_optimization.py: new test `test_linear_svr_uses_extended_convergence_budget` asserts defaults, optimized params, and the constructed model all use `max_iter == 10_000`.

4fa75b0 — fix(models): tighten kernelridge_reg search space to

reduce overfitting **Type:** fix | **Date:** 2026-07-20

Description: Constrains the KernelRidge regression hyperparameter search space to curb overfitting in the $p \gg n$ genomics regime: very small `alpha` let RBF kernels interpolate the training set exactly, and large `gamma` degenerated the kernel matrix toward the identity. A `linear` kernel option is also added.

Code:

```
- kernel = trial.suggest_categorical('kernel', ['rbf', 'laplacian', 'polynomial', 'sigmoid'])
+ kernel = trial.suggest_categorical('kernel', ['linear', 'rbf', 'laplacian', 'polynomial',
+ 'sigmoid'])

params = {
-     'alpha': trial.suggest_float('alpha', 1e-6, 10.0, log=True),
+     # lower bound raised to 1e-2: with p >> n, too-small alpha lets RBF
+     # kernels interpolate the training set (overfitting)
+     'alpha': trial.suggest_float('alpha', 1e-2, 10.0, log=True),
```

```

        'kernel': kernel,
    }

    if kernel in ['rbf', 'laplacian', 'polynomial', 'sigmoid']:
-     params['gamma'] = trial.suggest_float('gamma', 1e-6, 10.0, log=True)
+     # gamma capped at 1.0 to avoid a near-identity kernel matrix
+     params['gamma'] = trial.suggest_float('gamma', 1e-6, 1.0, log=True)

```

Changes: - gpse/models/regression_model_optimizer.py: kernelridge_reg search space now includes the linear kernel. - alpha lower bound raised from 1e-6 to 1e-2 (log-uniform up to 10.0 unchanged). - gamma upper bound capped at 1.0 (was 10.0) for rbf/laplacian/polynomial/sigmoid kernels. - Single-file change (5 insertions, 4 deletions); only the Optuna search space is affected, defaults untouched.

c0c791c — fix(train): scope CatBoost outputs to results directory

Type: fix | **Date:** 2026-07-20

Description: Stops CatBoost from littering the working directory with its catboost_info training logs. Both model optimizers accept a new catboost_train_dir; when set, CatBoost writes logs there, and when unset, CatBoost file writing is disabled entirely. Training and classification tasks now point it at <results_dir>/catboost_info.

Code:

```

+ # gpse/models/regression_model_optimizer.py (same pattern in classification optimizer)
    def __init__(self, random_seed=None, n_threads=1, random_state=None,
+                 catboost_train_dir: Optional[str] = None,
    ):
+     self.catboost_train_dir = catboost_train_dir # CatBoost train_dir (catboost_info output)

    elif model_name == 'catboost_reg':
        from catboost import CatBoostRegressor
        params['thread_count'] = self.n_threads
+     if self.catboost_train_dir:
+         params.setdefault('train_dir', self.catboost_train_dir)
+     else:
+         params.setdefault('allow_writing_files', False)
    return CatBoostRegressor(**params)

# gpse/train/predictor.py
self.model_optimizer = RegressionModelOptimizer(
-     random_seed=random_seed, n_threads=n_threads
+     random_seed=random_seed, n_threads=n_threads,
+     catboost_train_dir=str(self.results_dir / "catboost_info"),
)

```

Changes: - gpse/models/regression_model_optimizer.py and gpse/models/classification_model_optimizer.py: new catboost_train_dir constructor argument; catboost_reg/catboost_clf creation sets train_dir or falls back to allow_writing_files=False.

- gpse/train/predictor.py: regression optimizer is constructed with

catboost_train_dir=<results_dir>/catboost_info. - gpse/tasks/classification.py: classification optimizer likewise scoped to <results_dir>/catboost_info. - tests/test_train_pipeline_optimization.py: 62 new test lines covering the CatBoost output-directory behavior.

6835627 — docs(wiki): add wiki framework for CLI subcommands and

API reference **Type:** docs | **Date:** 2026-07-20

Description: Adds a docs/wiki/ documentation set covering the software overview, the convert/train/predict subcommands, configuration files, and a Python API reference, with an index README and quick-start examples. The same commit also silences a pandas FutureWarning about fillna downcasting in the training report generator.

Code:

```
<!-- docs/wiki/README.md - new wiki index -->
| 章节 | 内容 |
|-----|-----|
| [01. 软件总览](01-overview.md) | 软件作用、核心特性、整体架构、依赖 |
| [02. `gpse convert`](02-cli-convert.md) | 数据转换子命令: pipeline / qc / recode / deps 四种模式 |
| [03. `gpse train`](03-cli-train.md) | 训练子命令: 超参优化、重复交叉验证、Stacking 集成 |
| [04. `gpse predict`](04-cli-predict.md) | 预测子命令: 特征对齐、覆盖度校验、表型预测 |
| [05. 配置文件](05-configuration.md) | `gpse.yaml` / 包内置配置 / TOPSIS 配置 |
| [06. API 参考](06-api-reference.md) | 各模块公开类与函数签名 |

# gpse/train/_results.py - silence pandas FutureWarning on fillna downcasting
-markdown_rows = leaderboard[table_columns].fillna("").astype(str)
+markdown_rows = leaderboard[table_columns].fillna("").infer_objects(copy=False).astype(str)
```

Changes: - New docs/wiki/ framework (7 files, 496 lines): README.md index, 01-overview.md, 02-cli-convert.md, 03-cli-train.md, 04-cli-predict.md, 05-configuration.md, 06-api-reference.md. - Wiki documents the four convert modes (pipeline/qc/recode/deps), the train workflow (Optuna optimization, repeated CV, stacking), predict feature alignment and coverage checks, and config file layout, plus quick-start command examples. - gpse/train/_results.py: one-line fix adding .infer_objects(copy=False) after fillna("") in _markdown_report to silence the pandas down-casting FutureWarning.

1c51409 — fix(utils): skip correlation on constant inputs to

silence ConstantInputWarning **Type:** fix | **Date:** 2026-07-20

Description: Correlation coefficients are undefined when either input array is constant, causing scipy to emit ConstantInputWarning and return NaN. calculate_metrics() now detects constant inputs up front and reports 0.0 instead of relying on exception handling to swallow the warning.

Code:

```
def calculate_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> Dict[str, float]:
    """Calculate all performance metrics."""
    try:
        pearson_corr = pearsonr(y_true, y_pred)[0]
        if np.isnan(pearson_corr):
            pearson_corr = 0.0
    except:
        # Constant inputs make correlation coefficients undefined (scipy emits
        # ConstantInputWarning and returns NaN); treat them as 0.0 instead.
        if np.ptp(y_true) == 0 or np.ptp(y_pred) == 0:
            pearson_corr = 0.0
            spearman_corr = 0.0
        else:
            try:
                pearson_corr = pearsonr(y_true, y_pred)[0]
                if np.isnan(pearson_corr):
                    pearson_corr = 0.0
            except:
                pearson_corr = 0.0
            try:
                spearman_corr = spearmanr(y_true, y_pred)[0]
            ...
```

Changes: - Constant `y_true` or `y_pred` (checked via `np.ptp(...) == 0`) now short-circuits Pearson and Spearman to 0.0 without calling scipy, silencing `ConstantInputWarning` in `gpse/utils/genomic_utils.py`.
 - The NaN/exception fallback to 0.0 is preserved for non-constant inputs, so behavior for normal arrays is unchanged.

3fe1c39 — style(log): collapse fold results into a single line

Type: style | **Date:** 2026-07-20

Description: Per-fold metric logging previously emitted 7–8 separate `info` lines per fold, flooding the log during parallel runs. Each fold's results are now rendered as a single compact line with 4-decimal precision.

Code:

```
- task_logger.info(f"Fold {fold_idx + 1} results:")
- task_logger.info(f"  Train Pearson: {train_metrics['pearson']:.6f}")
- ...
- task_logger.info(f"  Training time: {training_time:.2f}s")
+ task_logger.info(
+     f"Fold {fold_idx + 1} | "
+     f"Train r={train_metrics['pearson']:.4f} p={train_metrics['spearman']:.4f} | "
+     f"Val r={val_metrics['pearson']:.4f} p={val_metrics['spearman']:.4f} | "
+     f"Test r={test_metrics['pearson']:.4f} p={test_metrics['spearman']:.4f} "
+     f"MSE={test_metrics['mse']:.4f} | "
+     f"{training_time:.2f}s"
+ )
```

Changes: - `GenomicClassifier.log_classification_results()` in `gpse/tasks/classification.py` now emits one line per fold (Fold N | Train acc=... f1=... | Val ... | Test ... [auc=...]) instead of six to seven lines. - Regression fold logging in `gpse/train/_fold_training.py` (`_log_fold_results`) likewise collapses to a single line including training time; metric precision reduced from 6 to 4 decimals. - No functional change to metrics — logging format only.

cc4820e — style(log): tag fold/repeat log lines with model name

and compact summaries **Type:** style | **Date:** 2026-07-20

Description: Fold and repeat-average log lines are now prefixed with a `model R<repeat> F<fold>` tag so output from concurrently running models stays attributable, and the multi-line repeat-average block is condensed into a single tagged summary line.

Code:

```
+ tag = f"{model_name} R{repeat_idx + 1} F{fold_idx + 1}"
+ if self.task_type == "classification":
+     self.genomic_classifier.log_classification_results(
-         fold_idx, train_metrics, val_metrics, test_metrics, task_logger
+         fold_idx, train_metrics, val_metrics, test_metrics, task_logger, tag=tag
+     )
+ ...
+     line = (
+         f"{tag} avg | "
+         f"Train acc={metrics['avg_train_accuracy']:.4f}±{metrics['std_train_accuracy']:.4f} "
+         f"f1=... | Val acc=... f1=... | Test acc=... f1=..."
+     )
+     line += f" | {metrics['avg_training_time']:.2f}s"
+     task_logger.info(line)
```

Changes: - `log_classification_results()` in `gpse/tasks/classification.py` gains an optional `tag` parameter that replaces the default `Fold N` prefix. - `_log_fold_results()` and

_calculate_fold_average_metrics() in gpse/train/_fold_training.py accept model_name/repeat_idx and tag every line, e.g. xgboost_reg R1 F3 | Train r=... - Repeat-average output collapses from 9 lines to a single line with mean±std per split (classification and regression); test MSE now formats with :.4g. - The “Training fold N/M...” message in gpse/train/_repeat_training.py is demoted from info to debug.

409b152 — fix(batch): stop remaining tasks on Ctrl+C interrupt

Type: fix | **Date:** 2026-07-20

Description: Previously, hitting Ctrl+C during a batch run was logged as an ordinary task failure and the runner continued with the remaining queue. The batch runner now propagates the interrupt to the child process and aborts the whole batch on return code 130 / -SIGINT.

Code:

```
-         return_code = process.wait()
+         try:
+             return_code = process.wait()
+         except KeyboardInterrupt:
+             process.terminate()
+             return_code = process.wait()
+
+     ...
+     # Ctrl+C (return code 130 / -SIGINT) stops the whole batch
+     if return_code in (130, -2):
+         print(f"\n█ 检测到用户中断, 终止剩余 {len(tasks) - i} 个任务")
+         break
```

Changes: - run_task() in batch/batch_genomic_prediction.py catches KeyboardInterrupt around process.wait(), terminates the child process, and reaps its exit code. - A return code of 130 or -2 is reported as a user interrupt rather than a task failure. - The main loop breaks out of the task queue on interrupt, printing how many remaining tasks were cancelled.

bcacd92 — feat(train): recycle leftover thread budget into

n_jobs; demote fold logs to debug **Type:** feat | **Date:** 2026-07-20

Description: derive_parallelism_from_threads() previously left n_jobs at 1, wasting cores whenever model-level and repeat-level parallelism did not exhaust the --threads budget. Leftover budget is now recycled into per-worker n_jobs, and per-fold log lines move to DEBUG level so parallel runs stay readable.

Code:

```
+     # Give any leftover budget to per-worker threads so the --threads target
+     # is actually used (e.g. threads=80, 15 models, 2 repeats would otherwise
+     # only use 15 * 2 = 30 workers and waste the remaining cores).
+     if n_jobs == 1:
+         used = derived_max_workers * derived_repeat_workers
+         derived_n_jobs = max(1, threads // used)
+
+     return derived_n_jobs, derived_max_workers, derived_repeat_workers
```

Changes: - derive_parallelism_from_threads() in gpse/utils/paralle.py computes n_jobs = threads // (max_workers * repeat_workers) when n_jobs was not explicitly set (e.g. --threads 80 with 15 models × 2 repeats now yields n_jobs=2, using 60 cores instead of 30). - log_classification_results() gains a level parameter; fold result lines in gpse/train/_fold_training.py are emitted at DEBUG (use -l DEBUG to see). - Explicitly set n_jobs/max_workers/repeat_workers values are still preserved. - tests/test_parallelism.py updated: single-repeat case now expects n_jobs == 7 (100 scenario).

5c811cf — fix(train): drop invalid n_jobs for HistGBR; colorize

repeat logs; route stage rule through loguru **Type:** fix | **Date:** 2026-07-20

Description: Three training-side fixes plus docs: HistGradientBoostingRegressor no longer receives an invalid n_jobs parameter (HGBR parallelism is governed by OpenMP/BLAS env vars), repeat log headers are colorized via loguru markup, and the stage separator is routed through loguru instead of a separate Rich console so it cannot interleave with async log records.

Code:

```
elif model_name == 'histgradientboost_reg':
    from sklearn.ensemble import HistGradientBoostingRegressor
-     params['n_jobs'] = self.n_threads
+     # HGBR has no n_jobs parameter; its parallelism is governed by
+     # the OpenMP/BLAS thread environment variables.
    return HistGradientBoostingRegressor(**params)

-def _log_stage(title: str) -> None:
-     """Print a pretty stage separator to terminal and log to file."""
-     if _console is not None:
-         _console.rule(f"[bold blue]{title}[/bold blue]")
-     main_logger.info(title)
+def _log_stage(title: str, width: int = 76) -> None:
+     """Log a stage separator through loguru."""
+     pad = max(width - len(title) - 2, 4)
+     left = pad // 2
+     right = pad - left
+     main_logger.info(f"{'-' * left} {title} {'-' * right}")

# requirements.yaml
-e git+ssh://git@github.com:xsx123123/GPSE.git@bcacd9241e8c8eb75f2ef4149e69607624178dc4#egg=gpse
```

Changes: - gpse/models/regression_model_optimizer.py: removes params['n_jobs'] = self.n_threads for histgradientboost_reg, fixing an invalid-kwarg error against sklearn's HGBR. - gpse/train/_fold_training.py and gpse/train/_repeat_training.py: repeat headers, train/test sizes, default-param dumps, and avg summary lines now use logger.opt(colors=True) with loguru color markup (<cyan>, <green>, <yellow>, , <d>). - gpse/train/workflow.py: _log_stage() drops the optional Rich Console.rule() path and renders a fixed-width – rule through main_logger, keeping it serialized with enqueued log messages. - README.md: adds a 2026-07-20 “Recent Updates” entry summarizing the logging overhaul, parallelism recycling, batch Ctrl+C handling, and warning silencing. - requirements.yaml: editable gpse pin bumped from commit 737abae to bcacd92.

d7f863e — fix(log): use colored stderr sink for default console

style so ANSI colors render **Type:** fix | **Date:** 2026-07-20

Description: The default console log style used a Rich RichHandler, which cannot render loguru <color> markup embedded in messages (the tags are stripped for non-colorize sinks), so colored log lines lost their ANSI colors. The default style was switched to a colorized loguru sys.stderr sink that mimics the Rich look.

Code:

```
-     handler = RichHandler(
-         show_time=True,
-         omit_repeated_times=False,
-         show_path=False,
-         markup=True,
-         rich_tracebacks=True,
-         log_time_format="[%X]",
```

```

+         # Default style: colored stderr sink. A RichHandler cannot render
+         # loguru ``<color>`` markups in messages (they get stripped for
+         # non-colorize sinks), so use a colored stream that mimics the
+         # Rich look: "[HH:mm:ss] LEVEL message".
+         fmt = (
+             "<green>[time:HH:mm:ss]</green> "
+             "<level>{level: <8}</level> "
+             "{message}"
+         )
-         fmt = "{message}"
+         logger.add(sys.stderr, format=fmt, level=log_level, colorize=True, enqueue=True)
+         return

```

Changes: - In `gpse/utils/log_utils.py _add_console_handler()`, the default console style now registers a `colorize=True` stderr sink with a `[HH:mm:ss] LEVEL message` format instead of a `RichHandler`. - The plain/simplified styles and file logging paths are unchanged.

e0d0d6f — Revert “fix(log): use colored stderr sink for default console style so ANSI colors render” **Type:** fix | **Date:** 2026-07-20

Description: Reverts commit `d7f863e`, restoring the `RichHandler`-based default console style. The colored-stderr-sink change was rolled back (per the commit message, it is a straight revert of the immediately preceding logging fix).

Code:

```

-         fmt = (
-             "<green>[time:HH:mm:ss]</green> "
-             "<level>{level: <8}</level> "
-             "{message}"
+         handler = RichHandler(
+             show_time=True,
+             omit_repeated_times=False,
+             show_path=False,
+             markup=True,
+             rich_tracebacks=True,
+             log_time_format="%X",
+         )
-         logger.add(sys.stderr, format=fmt, level=log_level, colorize=True, enqueue=True)
-         return
+         fmt = "{message}"

```

Changes: - `gpse/utils/log_utils.py` is returned byte-for-byte to its state before `d7f863e`: default console style again uses `RichHandler` with `markup=True` and `rich_tracebacks=True`, emitting bare `{message}`.

6298af4 — chore(log): reduce noisy separators and fix wrapped log lines **Type:** chore | **Date:** 2026-07-20

Description: Cleans up console log noise during training runs: the per-repeat `=*50` separator banner is merged into a single concise “Model X - Repeat i/N: starting training” line, redundant trailing separators at model/stacking boundaries are dropped, and messages that began with a literal `\n` are split into separate empty-line calls so titles no longer wrap onto the line after the INFO prefix.

Code:

```

-     sep = "=" * 50
-     task_logger.opt(colors=True).info(f"<cyan>{sep}</cyan>")
-     task_logger.opt(colors=True).info(

```

```

-         f"<b><cyan>Model {model_name}</cyan></b> "
-         f"<yellow>- Repeat {repeat_idx + 1}/{self.n_repeats}</yellow>"
-     )
-     task_logger.opt(colors=True).info(f"<cyan>{sep}</cyan>")
-     main_logger.opt(colors=True).info(
-         f"Starting training Model:<cyan>{model_name}</cyan> "
-         f"Repeat:<yellow>{repeat_idx + 1}</yellow>"
+         f"Model <b><cyan>{model_name}</cyan></b> "
+         f"<yellow>- Repeat {repeat_idx + 1}/{self.n_repeats}</yellow>: starting training"
    )
+     if task_logger is not main_logger:
+         main_logger.opt(colors=True).info(
+             f"Model <b><cyan>{model_name}</cyan></b> "
+             f"<yellow>- Repeat {repeat_idx + 1}/{self.n_repeats}</yellow>: starting training"
+         )
-     self.logger.info(f"\n--- Processing trait: {trait} ---")
+     self.logger.info("")
+     self.logger.info(f"--- Processing trait: {trait} ---")

```

Changes: - gpse/train/_repeat_training.py: collapses the three-line per-repeat banner into one log line, only mirrors it to main_logger when task and main loggers differ, and removes the trailing =70 separator after the “Starting N repeated training runs” message. - gpse/train/_pipeline.py: removes the redundant second =70 separator around the “Starting Stacking ensemble” header. - Leading-\n split applied to trait-processing headers in gpse/convert/processor.py, phenotype headers in gpse/convert/genotype_matrix.py, fold-ensemble metric blocks in gpse/train/_ensemble.py, the early-stopping notice in gpse/train/_optimization.py, and the comparison-table header in gpse/ utils/genomic_utils.py. - No behavior changes; pure log formatting.

1ed2b1b — feat(convert): add Azodi 2019 conversion mode (-101
encoding, min-max phenotype scaling) Type: feat | **Date:** 2026-07-20

Description: Adds support for the Azodi et al. 2019 preprocessing convention: genotypes can now be encoded as centered additive -1/0/1 (in addition to the default 0/1/2), and phenotypes can be min-max normalized to [0, 1] (in addition to z-score standardization). New CLI options --geno-encoding and --pheno-scale expose both, with --standardize-phenotype kept as a legacy alias for --pheno-scale zscore.

Code:

```

+ # Centered additive encoding, Azodi et al. 2019 style: [-1, 0, 1] = [aa, Aa, AA]
+ GENO_DICT_CENTERED = {
+     '00': '-1', # Homozygous reference (aa)
+     '01': '0',  # Heterozygous (Aa)
+     '10': '0',  # Heterozygous (Aa)
+     '11': '1',  # Homozygous alternate (AA)
+ }
+ GENO_ENCODINGS = {'012': GENO_DICT, '-101': GENO_DICT_CENTERED}

+ def minmax_normalize_phenotype(pheno_df, trait_col, *, logger=None):
+     """Apply min-max normalization to a phenotype column, scaling to [0, 1]."""
+     y = pd.to_numeric(pheno_df[trait_col], errors='coerce')
+     ...
+     pheno_normalized[trait_col] = (y - min_val) / (max_val - min_val)
+     scaler_params = {'min': min_val, 'max': max_val, 'applied': True,
+         'method': 'minmax', 'trait': trait_col}

conv.add_argument(
    "--geno-encoding", default="012", choices=["012", "-101"],
    help="Genotype additive encoding: '012' = 0/1/2 (default), '-101' = -1/0/1 ...")

```

```
pheno.add_argument(
    "--pheno-scale", default=None, choices=["none", "zscore", "minmax"],
    help="Phenotype scaling mode ... Overrides --standardize-phenotype.")
```

Changes:

- gpse/convert/genotype_matrix.py: adds GENO_DICT_CENTERED/GENO_ENCODINGS; convert_to_matrix() and process_snp_dir() accept geno_encoding ("012" default, "-101" centered; unknown values raise ValueError). Missing genotypes still map to 3.
- gpse/convert/phenotype.py: new minmax_normalize_phenotype() scaling to [0, 1]; scaler param dicts now carry a method tag (zscore/minmax) for later reversal.
- gpse/convert/processor.py: _process_single_trait() takes pheno_scale (none/zscore/minmax) instead of a boolean; --standardize-phenotype maps to zscore for backward compatibility; scaler JSON and summary logs report the chosen method.
- gpse/utils/cli_display.py: new --geno-encoding {012,-101} and --pheno-scale {none,zscore,minmax} options on the convert subcommand; gpse/convert/workflow.py forwards both to the pipeline.
- Adds tests/test_convert_readme_mode.py (155 lines) covering the new conversion mode.

a3712aa — chore(log): shorten long paths in logs and fix pandas

FutureWarning **Type:** chore | **Date:** 2026-07-20

Description: Introduces a shorten_path() helper that collapses the home directory to ~ and leading components to .../ so long file paths stay on a single log line, applies it to result/report/summary path logs, and fixes a pandas silent-downcasting FutureWarning in the markdown report generator.

Code:

```
+def shorten_path(path, max_len: int = 80) -> str:
+    """Abbreviate a filesystem path for one-line console logs."""
+    p = str(path)
+    home = str(Path.home())
+    if home != "/" and p.startswith(home + "/"):
+        p = "~" + p[len(home):]
+    if len(p) <= max_len:
+        return p
+    parts = p.split("/")
+    tail = parts[-1]
+    kept = tail
+    for part in reversed(parts[:-1]):
+        candidate = part + "/" + kept
+        if len(".../" + candidate) > max_len:
+            break
+        kept = candidate
+    if len(".../" + kept) <= max_len:
+        return ".../" + kept
+    return "..." + p[-(max_len - 1):]

- markdown_rows = leaderboard[table_columns].fillna("").infer_objects(copy=False).astype(str)
+ markdown_rows = leaderboard[table_columns].where(lambda df: df.notna(), "").astype(str)
```

Changes:

- gpse/utils/log_utils.py: new public shorten_path(path, max_len=80) helper (home → ~, then ellipsis-collapsed leading dirs, keeping the filename visible).
- Applied to path logs in gpse/train/_pipeline.py (hold-out reports dir, result bundle paths), gpse/train/_repeat_training.py (repeat results, representative model, model summary paths), and gpse/utils/genomic_utils.py (comparison table path).
- gpse/train/_results.py: replaces fillna("") + infer_objects(copy=False) with where() in the markdown leaderboard table to silence the pandas FutureWarning.
- No output-file or behavior changes beyond log rendering and warning suppression.

ceafb6c — feat(train): save final hold-out predictions from
CV-TOPSIS top model **Type:** feat | **Date:** 2026-07-20

Description: The training-side CV TOPSIS ranking now always runs when ≥ 2 models are trained, regardless of `--use-stacking` (stacking just consumes the top-N of that ranking). From the TOPSIS rank-1 model's selected repeat, hold-out predictions are exported to `reports/final_predictions.csv` (ID, observed, predicted), averaging fold test predictions for regression or majority-voting for classification, and reversing phenotype standardization when applied. Also bumps the version to 0.0.2 and adds 6-species validation documentation and driver scripts.

Code:

```
+def _write_topsis_final_predictions(results_dir, all_model_results, X, task_type):
+    """Persist hold-out predictions from the CV-TOPSIS top-ranked model."""
+    topsis_simple_path = results_dir / "model_comparison_cv_topsis_simple.csv"
+    if topsis_simple_path.exists():
+        ranking = pd.read_csv(topsis_simple_path).sort_values("TOPSIS_Rank")
+        best_model = ranking.iloc[0]["Model"]
+    else: # fallback: CV mean/std ranking
+        ...
+    repeat_idx = int(selection.get("selected_repeat_idx", 0))
+    predictions_file = results_dir / best_model / f"repeat_{repeat_idx + 1}" /
+    "all_predictions.json"
+    ...
+    if task_type == "classification":
+        predicted = pd.DataFrame(predicted_stack).mode().iloc[0].to_numpy()
+    else:
+        predicted = predicted_stack.mean(axis=0)
+    if scaler.get("applied"): # reverse phenotype standardization
+        observed = observed * scaler["std"] + scaler["mean"]
+        predicted = predicted * scaler["std"] + scaler["mean"]
+    pd.DataFrame({"ID": X.index[indices].astype(str),
+                 "observed": observed,
+                 "predicted": predicted}).to_csv(reports_dir / "final_predictions.csv",
+        index=False)

-    if use_stacking and len(all_model_results) >= 2:
+    if len(all_model_results) >= 2:
+        ...
-        selected_models_for_stacking = pd.read_csv(topsis_simple_path).head(top_n_models)
+    ["Model"].tolist()
+    topsis_ranked_models = pd.read_csv(topsis_simple_path)["Model"].tolist()
+    if use_stacking:
+        selected_models_for_stacking = topsis_ranked_models[:top_n_models]
```

Changes: - `gpse/train/_pipeline.py`: new `_write_topsis_final_predictions()` writes `reports/final_predictions.csv` from the TOPSIS rank-1 model; CV-TOPSIS ranking (`model_comparison_cv_topsis_simple.csv`) is computed even without `--use-stacking`, and `stacking_selected_models.json` is only written when stacking is enabled. - Predictions come from the best model's `repeat_{N}/all_predictions.json` test entries; falls back to CV mean/std ranking and warns/skips gracefully when TOPSIS output or prediction files are missing. - Phenotype scaler from `split_manifest.json` is reversed on observed/predicted values when standardization was applied. - Version bumped to 0.0.2 in `pyproject.toml`, `gpse/config/default.yaml`, and `gpse/config/software.yaml`. - Adds `docs/6species_reproduction_validation.md`, `docs/6species_input_sha256.txt`, `docs/6species_result_baseline.tsv`, and validation driver scripts under `tests/validation/`.

c4b0ce0 — feat(batch): add gpse batch subcommand for multi-trait

YAML-driven training **Type:** feat | **Date:** 2026-07-20

Description: Introduces the new `gpse batch` workflow, which trains models for multiple traits from a single YAML config file. A shared `defaults` section is merged with per-trait overrides, translated into `gpse train` arguments via `argparse` introspection of the train parser, and executed sequentially — one trait's failure does not stop the batch, and a per-trait summary is printed at the end.

Code:

```
+def _train_option_actions() -> dict[str, argparse.Action]:
+    """Map every ``gpse train`` long option name to its argparse action."""
+    from gpse.train.workflow import _build_parser
+    parser = _build_parser()
+    actions: dict[str, argparse.Action] = {}
+    for action in parser._actions:
+        for option in action.option_strings:
+            if option.startswith("--"):
+                actions[option[2:]] = action
+    return actions
+
+def build_trait_argv(defaults, trait, actions=None) -> list[str]:
+    merged = {**defaults, **{k: v for k, v in trait.items() if k not in RESERVED_KEYS}}
+    ...
+    if not merged.get("results_dir"):
+        merged["results_dir"] = str(Path(str(results_root)) / name)
+    merged["target_trait"] = name
+    ...
+
+def run_batch(config_path, dry_run=False) -> int:
+    """Run ``gpse train`` for every enabled trait; return 0 when all succeed."""
+    ...
+    if exit_code == 130:
+        main_logger.warning("Batch interrupted by user (Ctrl+C).")
+        outcomes.append((name, "interrupted"))
+        return 130
+    outcomes.append((name, "ok" if exit_code == 0 else f"failed({exit_code})"))
```

Changes: - New package `gpse/batch/` with `cli.py` (flags `--config`, `--dry_run`) and `runner.py` (config loading, argv construction, sequential trait execution). - YAML schema: `defaults` for shared `gpse train` options plus `results_root`; traits list with name, per-trait overrides, `results_dir`, and enabled to skip a trait. - Option translation handles `store_true`, `BooleanOptionalAction` (emits `--no-<key>`), and list-valued (`nargs="+"`) train flags; unknown keys raise a validation error. - Registered `batch` in the root command router (`gpse/cli.py`) and updated the no-command / unknown-command help text. - Added `batch/batch_config.example.yaml` template and 15 unit tests in `tests/test_batch_runner.py`; documented in `README.md` and `docs/readme_cn.md`. - Added no-CV and maize_HT validation driver scripts under `tests/validation/`.

2026-07-21

e19517d — feat(batch): unify batch logging with train style and

pretty-print dry-run commands **Type:** feat | **Date:** 2026-07-21

Description: Polishes the `gpse batch` console experience: it now initializes the standard GPSE logger so batch output matches the `[HH:MM:SS] INFO ...` style of `gpse train`, and `--dry_run` renders each trait's command as a multi-line, continuation-joined `gpse train` invocation grouped per flag.

Code:

```

+ from gpse.utils.log_utils import logger_init
+
+ logger_init(log_level=getattr(args, "log_level", "INFO"))

+def _format_train_command(argv: list[str]) -> str:
+    """Render a ``gpse train`` argv as a multi-line copy-pasteable command."""
+    groups: list[list[str]] = []
+    for token in argv:
+        if token.startswith("--") or not groups:
+            groups.append([token])
+        else:
+            groups[-1].append(token)
+    lines = ["gpse train \\"]
+    lines.extend(f"    {' '.join(group)} \\\" for group in groups[:-1])
+    lines.append(f"    {' '.join(groups[-1])}")
+    return "\n".join(lines)

    if dry_run:
-        print(f"gpse train {' '.join(argv)}")
+        print(f"[{index}/{len(runnable)}] {name}")
+        print(_format_train_command(argv))

```

Changes: - gpse/batch/cli.py: calls logger_init() so batch logs use the train-style format instead of the rich_color_ext box-style sink installed at import time. - gpse/batch/runner.py: new _format_train_command() groups each flag with its values on its own continuation line, keeping long paths intact and copy-paste friendly. - Dry-run output now prefixes each command with a [index/total] trait-name header. - Added per-species batch validation configs (tests/validation/*_batch_config.yaml) and runner scripts for the 6-species dataset (maize, rice, sorghum, soy, spruce, switchgrass).

1bfd689 — docs(readme): add PyPI install instructions and version

badge **Type:** docs | **Date:** 2026-07-21

Description: Updates both READMEs to reflect the PyPI release: adds a PyPI version badge and makes pip install gpse the recommended install path, demoting the Poetry workflow to “Install from Source”.

Code:

```

<a href="https://pypi.org/project/gpse/"></a>

```

```

### Install via pip (Recommended)

```

```

pip install gpse

```

Changes: - Added a shields.io PyPI version badge to README.md and docs/readme_cn.md. - Reordered install instructions: pip install gpse is now the recommended method; Poetry/pip install . moved under “Install from Source”.

08a226e — fix(train): work around scipy-OpenBLAS segfault in

KernelRidge solve **Type:** fix | **Date:** 2026-07-21

Description: Fixes intermittent whole-process crashes (which also killed the model process pool) in gblup_reg/kernelridge_reg fits inside scipy.linalg.solve, caused by the OpenBLAS build bundled with scipy wheels dereferencing a null pointer in multithreaded dposv/dtrsm kernels. A new

SafeKernelRidge pins the tiny Cholesky solve to a single BLAS thread; additionally, the pipeline now fails gracefully when every model fails instead of raising StopIteration.

Code:

```
+class SafeKernelRidge(KernelRidge):
+    """KernelRidge whose internal Cholesky solve runs single-threaded."""
+
+    def fit(self, X, y=None, sample_weight=None):
+        original_solve = linalg.solve
+
+        def single_threaded_solve(*args, **kwargs):
+            with threadpool_limits(limits=1):
+                return original_solve(*args, **kwargs)
+
+        linalg.solve = single_threaded_solve
+        try:
+            return super().fit(X, y=y, sample_weight=sample_weight)
+        finally:
+            linalg.solve = original_solve
+
+        elif model_name == 'kernelridge_reg':
+            from sklearn.kernel_ridge import KernelRidge
+            return KernelRidge(**params)
+        from gpse.models.safe_kernel_ridge import SafeKernelRidge
+        return SafeKernelRidge(**params)
+
+    if not all_model_results:
+        raise RuntimeError(
+            "All models failed to train; see the per-model errors above. "
+            "No comparison, ensemble, or reports can be produced."
+        )
```

Changes: - New gpse/models/safe_kernel_ridge.py: SafeKernelRidge subclasses sklearn's KernelRidge and wraps scipy.linalg.solve in threadpool_limits(limits=1) during fit (kernel-matrix computation stays multithreaded). - gpse/models/regression_model_optimizer.py: kernelridge_reg and gblup_reg now construct SafeKernelRidge instead of plain KernelRidge. - gpse/utils/genomic_utils.py: create_comparison_table warns and returns on empty results instead of raising StopIteration. - gpse/train/pipeline.py: run_all_models raises a clear RuntimeError when every model failed. - Added tests/test_safe_kernel_ridge.py with regression coverage.

b72ee41 — feat(batch): show single-line command and derived config

in dry-run **Type:** feat | **Date:** 2026-07-21

Description: Improves gpse batch --dry_run output: each trait's command is printed as one shlex-quoted, copy-pasteable single line (replacing the previous backslash-continuation format), and the dry run now previews the exact derived configuration the real run will use.

Code:

```
def _format_train_command(argv: list[str]) -> str:
-    """Render a ``gpse train`` argv as a multi-line copy-pasteable command."""
-    ...
-    return "\n".join(lines)
+    """Render a ``gpse train`` argv as a single-line copy-pasteable command."""
+    return "gpse train " + shlex.join(argv)
+
+def _dry_run_trait_summary(argv: list[str]) -> list[str]:
+    """Render the derived per-trait configuration shown by a real training run."""
```

```

+     args = _build_parser().parse_args(argv)
+     if args.threads is not None:
+         n_jobs, max_workers, repeat_workers = derive_parallelism_from_threads(
+             threads=args.threads, n_models=_resolve_model_count(args),
+             n_repeats=args.n_repeats, ...)
+     lines = [
+         f" Results directory: {args.results_dir}",
+         f" Random seed: {args.random_seed}",
+         f" Threads per model: {n_jobs}",
+         f" Model workers: {max_workers}",
+         f" Repeat workers: {repeat_workers}",
+         ...
+     ]
+     lines.append(" " + ", ".join(f"{var}={n_jobs}" for var in ModelConstants.thread_env_vars))

```

Changes: - gpse/batch/runner.py: `_format_train_command()` now uses `shlex.join` for a single-line command without backslash continuations. - New `_dry_run_trait_summary()` re-applies the `--threads` budget derivation from gpse train to preview results dir, seed, threads per model, model/repeat workers, folds, repeats, the default-params flag, and thread environment variables (e.g. `OMP_NUM_THREADS`). - Dry-run prints the summary lines after each trait command. - tests/test_batch_runner.py: extended dry-run test to assert single-line commands and the derived-config summary.

74c9b36 — feat: add gpse tools split subcommand for train/test

data splitting **Type:** feat | **Date:** 2026-07-21

Description: Adds a new top-level gpse tools command with a `split` subcommand that divides matched genotype and phenotype samples into disjoint train/test subsets (default 80/20), so the test set can later be fed to gpse predict. Supports CSV/Parquet/Feather I/O, stratified splitting for classification, and reproducible splits via `--seed`.

Code:

```

+def split_dataset(geno_file, pheno_file, out_prefix, test_ratio=0.2,
+                 seed=42, stratify_column=None, id_column=None) -> dict:
+    # Resolve ID column (auto-detect 'ID' or first column), intersect samples
+    common_ids = geno_ids & pheno_ids
+    if not common_ids:
+        raise ValueError("No common sample IDs between genotype and phenotype")
+    ...
+    train_idx, test_idx = train_test_split(
+        indices, test_size=test_ratio, random_state=seed,
+        stratify=stratify_labels,
+    )
+    ...
+    # Writes <prefix>_train_geno, _train_pheno, _test_geno, _test_pheno

```

Changes: - New gpse/tools/split.py: `split_dataset()` aligns genotype/phenotype on the sample-ID column (auto-detected as `ID` or the first column), splits with sklearn's `train_test_split`, and writes four output files (`<prefix>_train_geno`, `<prefix>_train_pheno`, `<prefix>_test_geno`, `<prefix>_test_pheno`) in the genotype file's format. - New gpse/tools/cli.py: gpse tools split CLI with flags `--geno`, `--pheno`, `--out-prefix`, `--test-ratio` (default 0.2), `--seed` (default 42), `--stratify COLUMN` for classification, and `--id-column`. - Registered tools in the root command router (gpse/cli.py) and root parser help (gpse/utis/cli_display.py). - Duplicate sample IDs are dropped (first occurrence kept); errors are raised for missing ID/stratify columns or empty ID intersection.

98411c8 — docs: move train_pipeline.md to docs/

Type: docs | **Date:** 2026-07-21

Description: A pure file relocation: the training-pipeline documentation was moved from the repository root into the docs/ directory to consolidate documentation in one place. No content changed.

Code:

```
train_pipeline.md => docs/train_pipeline.md | 0  
1 file changed, 0 insertions(+), 0 deletions(-)
```

Changes: - Renamed train_pipeline.md to docs/train_pipeline.md with no content modifications.

597f1d7 — feat(batch): merge per-trait summary tables across

traits after batch run **Type:** feat | **Date:** 2026-07-21

Description: After a gpse batch run finishes, the runner now concatenates each trait's summary CSVs (model_comparison*.csv and holdout summaries under reports/) into merged cross-trait tables under <results_root>/merged/, with a leading Trait column. Columns are unioned across traits so regression and classification traits with different metrics can share a table. This removes the need for ad-hoc scripts to compare traits.

Code:

```
+SUMMARY_FILES = (  
+    "model_comparison.csv",  
+    "model_comparison_cv.csv",  
+    "model_comparison_cv_topsis.csv",  
+    "model_comparison_cv_topsis_simple.csv",  
+    "reports/model_comparison_holdout.csv",  
+    "reports/holdout_metrics.csv",  
+)  
+  
+def merge_trait_results(  
+    trait_dirs: dict[str, Path],  
+    output_dir: str | Path,  
+) -> list[Path]:  
+    ...  
+    for relative in SUMMARY_FILES:  
+        frames = []  
+        for name, results_dir in trait_dirs.items():  
+            path = Path(results_dir) / relative  
+            if not path.exists():  
+                continue  
+            frame = pd.read_csv(path)  
+            frame.insert(0, "Trait", name)  
+            frames.append(frame)  
+        if not frames:  
+            continue  
+        merged = pd.concat(frames, ignore_index=True, sort=False)  
+        output_dir.mkdir(parents=True, exist_ok=True)  
+        out_path = output_dir / Path(relative).name  
+        merged.to_csv(out_path, index=False)  
+        written.append(out_path)  
+    return written  
  
# runner.py – after the trait loop:  
-     _log_summary(outcomes)  
+     _log_summary(outcomes, dry_run=dry_run)
```

```
+   if not dry_run:
+       _merge_batch_results(defaults, runnable, outcomes)
```

Changes: - New module `gpse/batch/merge.py` with `merge_trait_results()`, which merges the six known summary tables (skipping missing files, inserting a `Trait` column, unioning columns).

- `gpse/batch/runner.py`: added `_merge_batch_results()` that only

merges traits with `ok` status; output goes to `<results_root>/merged/`, or to the common parent of per-trait `results_dir` overrides when no `results_root` is set. Merge failures are logged as warnings and never fail the batch. - Dry-run output now goes through `loguru` instead of `print()`, with a highlighted “DRY-RUN” heading and centered section headers (`SECTION_WIDTH = 70`). - `batch/batch_config.example.yaml` documents the automatic merge behavior in a comment. - Extended `tests/test_batch_runner.py` with coverage for the new merge and logging behavior.

a35b8cf — chore(validation): run all 15 regression models in

6-species batch configs and add README **Type:** chore | **Date:** 2026-07-21

Description: Expanded the six-species validation batch configs to exercise every supported regression model (15 instead of 4), and added a `tests/validation/README.md` (in Chinese) documenting the validation suite, environment requirements, and how to run each species batch.

Code:

```
# tests/validation/maize_batch_config.yaml
- models: [rf_reg, xgboost_reg, lightgbm_reg, gblup_reg]
+ models:
+   - elasticnet_reg
+   - gbd_t_reg
+   - svr_reg
+   - mlp_reg
+   - knn_reg
+   - rf_reg
+   - xgboost_reg
+   - adaboost_reg
+   - lightgbm_reg
+   - catboost_reg
+   - kernelridge_reg
+   - gblup_reg
+   - histgradientboost_reg
+   - sg_d_reg
+   - lasso_reg
+   ...
- - name: YLD
-   models: [rf_reg, gblup_reg] # 该性状只跑两个模型
+ - name: YLD # 同样继承全部回归模型
```

Changes: - All six species configs (maize, rice, sorghum, soy, spruce, switchgrass under `tests/validation/`) now list the full set of 15 regression models in `defaults.models`. - Removed the per-trait two-model override for the maize `YLD` trait so it also inherits all regression models. - New `tests/validation/README.md` (260 lines, Chinese) describing the directory layout, prerequisites (`pip install -e .`, optional model dependencies, `threads: 60` tuning), a per-species config/script/trait table (with which trait enables stacking), and example commands including `--dry_run` and a loop over all six species.

445bab6 — chore(release): bump to 0.0.3a1 with validation suite

and doc updates **Type:** release | **Date:** 2026-07-21

Description: Release bump to 0.0.3a1. Alongside the version change, the validation suite gained batch configs and runner scripts for three cucurbit species (cucumber, melon, watermelon) and the lettuce MAGIC population (3D traits plus GWAS significant-SNPs sets), and the README/wiki documentation was refreshed — the wiki pages were translated to English and updated for the batch runner, the gpse tools split, and the implemented predict workflow.

Code:

```
# pyproject.toml
-version = "0.0.2"
+version = "0.0.3a1"

# tests/validation/cucumber_batch_config.yaml (new)
defaults:
  geno_file: /data/xiaoliu/gpse_3species/cucumber/geno/cucumber_geno.csv
  task_type: regression
  use_default_params: true
  test_size: 0.3
  n_splits: 5
  n_repeats: 2
  random_seed: 42
  use_same_test_set: true
```

Changes: - pyproject.toml: version 0.0.2 → 0.0.3a1. - New validation batch configs in tests/validation/: cucumber_batch_config.yaml (12 continuous traits), melon_batch_config.yaml, watermelon_batch_config.yaml, magic_3d_batch_config.yaml, and magic_gwas_batch_config.yaml, plus five corresponding *_batch.sh runner scripts. - tests/validation/README.md updated to document the new species/population configs. - README.md and docs/readme_cn.md: added a Documentation section with wiki links, documented the gpse tools split command, fixed stale predict/feature-manifest descriptions, and added a 0.0.3a1 changelog entry. - All docs/wiki/*.md pages translated to English and updated for the batch feature, the tools split, and the predict workflow.

2026-07-22

d3c9e5d — **perf: optimize hot-path bottlenecks in training and**
conversion pipeline **Type:** perf | **Date:** 2026-07-22

Description: Performance pass over hot paths in training and PED-to-matrix conversion: redundant per-fold index-map construction was hoisted out of loops, an $O(n \cdot m)$ list comprehension was replaced with `np.setdiff1d`, PED genotype encoding was vectorized, a duplicated `X_test` transform pass in the classification ensemble was removed, and an unnecessary genotype-data copy was dropped to reduce peak memory.

Code:

```
# gpse/convert/genotype_matrix.py – vectorized PED encoding
-     geno = row[6:]
-     encoded_geno = [geno_dict.get(g, '3') for g in geno]
-     sample_genotypes.append((sample_id, encoded_geno))
+     if raw_rows:
+         geno_array = np.array(raw_rows, dtype='U2')
+         lookup = np.vectorize(lambda g: geno_dict.get(g, '3'), otypes=['U2'])
+         encoded_array = lookup(geno_array)
+         sample_genotypes = list(zip(sample_ids, encoded_array.tolist()))

# gpse/train/_cv_manager.py & gpse/utils/genomic_utils.py
+     X_index_map = {idx: i for i, idx in enumerate(X.index)}
+     folds = []
```

```

        for fold_idx in range(self.n_splits):
            ...
            X_index_map = {idx: i for i, idx in enumerate(X.index)}

# gpse/utils/genomic_utils.py
-     train_indices = np.array([i for i in range(len(X)) if i not in test_indices])
+     train_indices = np.setdiff1d(np.arange(len(X)), np.asarray(test_indices), assume_unique=True)

# gpse/train/_data_io.py
-     X = geno_data.copy()
+     X = geno_data

```

Changes: - gpse/convert/genotype_matrix.py: PED parsing now collects raw rows and encodes the whole genotype array in one np.vectorize pass instead of per-element dict lookups. - gpse/train/_cv_manager.py and gpse/utils/genomic_utils.py: X_index_map dict is built once before the fold loop instead of once per fold. - gpse/train/_pipeline.py, gpse/train/_repeat_training.py, gpse/ utils/genomic_utils.py: train/test index subtraction now uses np.setdiff1d(..., assume_unique=True) (previously an O(n·m) comprehension or assume_unique=False). - gpse/train/_ensemble.py: classifica- tion ensemble prediction no longer transforms X_test twice per member — the transform is shared between the hard-prediction and probability passes. - gpse/train/_data_io.py: removed the geno_data.copy() when extracting the feature matrix, lowering peak memory during data loading.

386aa5a — feat(topsis): expand criteria config, add

-topsis_config CLI option **Type:** feat | **Date:** 2026-07-22

Description: The TOPSIS model-ranking configuration was overhauled: the bundled topsis.yaml now lists all 11 available metrics per task type (with weight 0 marking reference-only criteria), users can supply their own criteria/weights YAML via a new --topsis_config flag on gpse train, and training startup logs now print both the active and reference-only criteria.

Code:

```

# gpse/train/workflow.py
+     stack_group.add_argument(
+         "--topsis_config",
+         type=str,
+         default=None,
+         help=(
+             "Path to a custom TOPSIS criteria/weights YAML file. "
+             "Overrides the built-in gpse/config/topsis.yaml when provided"
+         ),
+     )

# gpse/config/_topsis_config.py
-     criteria.append(str(name))
-     criteria_types.append(str(criterion_type))
-     weights.append(weight)
+     if weight > 0:
+         criteria.append(str(name))
+         criteria_types.append(str(criterion_type))
+         weights.append(weight)
+
+     if not criteria:
+         raise ValueError(
+             f"TOPSIS task '{task_key}' has no criteria with weight > 0; ..."
+         )
+

```

```

+def get_all_topsis_criteria(task_type: str, user_config_path: str | None = None) -> list[dict]:
+    """Return the full criteria list (including weight=0) for display."""
# gpse/config/topsis.yaml – regression section now also lists (weight: 0):
# Test Spearman (+std), Validation Pearson (+std), Test MSE (+std),
# Ensemble Pearson (+std), Training Time (s)

```

Changes: - New CLI option `gpse train --topsis_config <yaml>` (`gpse/train/workflow.py`), threaded through `gpse/train/cli.py` and `gpse/train/predictor.py` into the predictor's `topsis_config` attribute. - `gpse/config/_topsis_config.py: _parse_topsis_task_config()` accepts an optional user config path and filters out weight-0 criteria; raises `ValueError` when no criterion has a positive weight. New `get_all_topsis_criteria()` helper for display. - `log_environment_settings()` now reports which TOPSIS config is in use (built-in vs. user-supplied) and logs active criteria (with type and weight) plus reference-only (weight=0) criteria. - `gpse/config/topsis.yaml`: expanded with header documentation (name/type/weight semantics, normalization note) and all available metrics per task type — classification gains Test F1, Test AUC, Validation Accuracy, Ensemble Accuracy, Training Time entries; regression gains Test Spearman, Validation Pearson, Test MSE, Ensemble Pearson, Training Time entries — all at weight 0 so default ranking behavior is unchanged. - `README.md` and `docs/wiki/05-configuration.md` document the TOPSIS config schema, weight semantics, and custom-config usage.

32d107a — feat(models): YAML-driven model registry with

-model_config option **Type:** feat | **Date:** 2026-07-22

Description: Replaces the hard-coded model definitions in the optimizers with a YAML-driven model registry (`gpse/config/models.yaml`) declaring all 15 regression and 6 classification models, so users can register or override models via a `--model_config` YAML file without code changes. Both optimizers are slimmed down to delegate model creation, defaults, and filtering to the new registry module.

Code:

```

# gpse/config/models.yaml – per-entry schema
models:
  - name: elasticnet_reg
    task: regression
    import_path: sklearn.linear_model.ElasticNet
    thread_strategy: none
    default_params:
      alpha: 1.0
      l1_ratio: 0.5
      random_state: "{random_seed}"
    search_space:
      - {name: alpha, type: float, low: 1.0e-8, high: 100.0, log: true}
      - {name: l1_ratio, type: float, low: 1.0e-5, high: 1.0, log: true}
      - {name: random_state, type: fixed, value: "{random_seed}"}

# gpse/config/_model_registry.py – inline Optuna search-space DSL
def _build_inline_param_func(search_space, context):
    def param_func(trial):
        params = {}
        for spec in search_space:
            if spec["type"] == "int":
                params[spec["name"]] = trial.suggest_int(spec["name"], spec["low"], spec["high"],
**kwargs)
            elif spec["type"] == "float":
                params[spec["name"]] = trial.suggest_float(spec["name"], spec["low"], spec["high"],
**kwargs)

```

```

        elif spec["type"] == "categorical":
            params[spec["name"]] = trial.suggest_categorical(spec["name"], spec["choices"])
        # ...
    return params
return param_func

+ # gpse/train/cli.py - user model config is deep-merged over the built-in registry
+ model_config = getattr(args, "model_config", None)
+ return len(RegressionModelOptimizer(
+     random_seed=42, n_threads=1, model_config_path=model_config
+ ).model_configs)

```

Changes: - New gpse/config/models.yaml (385 lines): name, task, import_path, thread_strategy, default_params, search_space / param_func per model; placeholders {random_seed}, {n_threads}, {n_classes} resolved at runtime. - New gpse/config/_model_registry.py: lazy import resolution, thread injection, inline search-space DSL compiled to Optuna callables, and builtin:/module: param_func references. - RegressionModelOptimizer and ClassificationModelOptimizer refactored to consume the registry (gpse/models/regression_model_optimizer.py, gpse/models/classification_model_optimizer.py). - New --model_config CLI option threaded through gpse/train/cli.py, gpse/train/predictor.py, gpse/train/workflow.py, and gpse/tasks/classification.py; new load_models_config() loader in gpse/utils/configuration.py. - Config YAML version synced to 0.0.3a1; .python-version set to gpse_new; README and docs/wiki/05-configuration.md document the registry.

d45d03a — gpse add framework

Type: docs | **Date:** 2026-07-22

Description: Adds the GPSE framework architecture diagram to the English and Chinese READMEs and updates the command screenshot image.

Code:

GPSE is a comprehensive, machine-learning-based pipeline for genomic selection and prediction. It provides end-to-end functionalities from raw genomic data (VCF/PLINK) preprocessing to hyperparameter optimization, model evaluation, TOPSIS ranking, and Stacking Ensemble prediction.

```

+ ![GPSE framework](docs/framework_v2.1.png)
+
+ ## 📖 Key Features

```

Changes: - New image docs/framework_v2.1.png (1.4 MB) embedded in README.md and docs/readme_cn.md. - Updated docs/command.png screenshot.

aa76e5e — fix(registry): make inline search-space param funcs

picklable for parallel workers **Type:** fix | **Date:** 2026-07-22

Description: Fixes a “Can’t pickle local object” failure at parallel-training submission time: the predictor (holding optimizers and their registry param functions) is pickled into ProcessPoolExecutor workers, but _build_inline_param_func returned a closure. The closure is replaced by a module-level InlineParamFunc callable class, and a regression test covers pickle round-trips for both optimizers and every registered param_func.

Code:

```

- def _build_inline_param_func(search_space, context):
-     """Compile a YAML search-space DSL into an Optuna param function."""
-     def param_func(trial) -> Dict[str, Any]:
+ class InlineParamFunc:
+     """Optuna param function compiled from a YAML search-space DSL.

```

```

+
+   Defined at module level (not as a closure) so that it can be pickled
+   and shipped to ``ProcessPoolExecutor`` model workers.
+   """
+   def __init__(self, search_space, context):
+       self.search_space = search_space
+       self.context = context
+
+   def __call__(self, trial) -> Dict[str, Any]:
+       params: Dict[str, Any] = {}
+       -       for spec in search_space:
+       +       for spec in self.search_space:
+           ...
+       return params
+   -   return param_func
+
+def _build_inline_param_func(search_space, context):
+   """Compile a YAML search-space DSL into an Optuna param function."""
+   return InlineParamFunc(search_space, context)

```

Changes: - gpse/config/_model_registry.py: closure replaced with the picklable `InlineParamFunc` class holding only `search_space` and `context`. - New tests/test_model_registry_pickle.py: pickle round-trips for `RegressionModelOptimizer`, `ClassificationModelOptimizer`, every registered `param_func`, and an Optuna `study.ask()` smoke test on `elasticnet_reg`. - Verified end-to-end with synthetic data: `elasticnet_reg/gbdt_reg/svr_reg/mlp_reg` complete under 4 parallel workers, with and without Optuna search.

2026-08-20

e7eff28 — fix(convert): allow -bfile as alternative genotype

source and clarify CLI help **Type:** fix | **Date:** 2026-08-20

Description: Makes `--bfile` (PLINK BED/BIM/FAM) a first-class direct input for `gpse convert` instead of requiring `--vcf`: validation now accepts exactly one of `--vcf`, `--bfile`, `--ped-file`, OR `--matrix-file` and rejects multiple sources with a clear error. The `convert` CLI help is reorganized into a “genotype source (choose one)” group, and a process-level aligned-data cache plus a `set_index` performance fix land in `gpse/train/_data_io.py`.

Code:

```

-   # Default pipeline — require core arguments.
+   # Default pipeline — require phenotype, output prefix, and exactly one
+   # supported genotype source. A PLINK bfile is a first-class direct input.
+   missing = []
-   if not args.vcf:
-       missing.append("--vcf")
+   genotype_sources = [args.vcf, args.bfile, args.ped_file, args.matrix_file]
+   provided_sources = [src for src in genotype_sources if src]
+   if not provided_sources:
+       missing.append(
+           "one genotype source: --vcf, --bfile, --ped-file, or --matrix-file"
+       )
+   elif len(provided_sources) > 1:
+       parser.error(
+           "Only one genotype source can be provided: "
+           "--vcf, --bfile, --ped-file, or --matrix-file."
+       )

```

```

- # Step 6: Set index and sort to ensure alignment
- geno_data.set_index(id_col, inplace=True)
- pheno_data.set_index(id_col, inplace=True)
+ # set_index() has pathological performance on ultra-wide genotype frames
+ # (94k SNP columns: ~370 s vs ~0.2 s for direct assignment + drop).
+ for frame in (geno_data, pheno_data):
+     frame.index = frame[id_col].to_numpy()
+     frame.index.name = id_col
+     frame.drop(columns=[id_col], inplace=True)

```

Changes: - gpse/convert/workflow.py: genotype-source validation accepts one of --vcf/--bfile/--ped-file/--matrix-file; multiple sources produce a parser error. - gpse/utils/cli_display.py: convert help regrouped into “genotype source (choose one)” and “required pipeline arguments” sections. - gpse/train/_data_io.py: new _ALIGNED_DATA_CACHE (keyed by file path + mtime) so multi-trait runs load/align geno/pheno once per process; index assignment replaces slow set_index on wide genotype frames. - gpse/train/cli.py: logs whether the stacking ensemble is enabled at startup. - New tests/test_convert_bfile.py with validation/parser tests; .gitignore now ignores .md files under tests/.

002ad72 — Merge branch ‘main’ of github.com:xsx123123/GPSE into main
Type: merge | **Date:** 2026-08-20

Description: Merges the remote main branch into local main, integrating the YAML model registry (models.yaml, _model_registry.py, --model_config), the TOPSIS config additions (gpse/config/topsis.yaml, _topsis_config.py), the pickle regression test, framework diagrams, and assorted train/convert tweaks (27 files, +1304/–665).

Changes: - Integrates: gpse/config/models.yaml + gpse/config/_model_registry.py registry, gpse/config/topsis.yaml + expanded _topsis_config.py, refactored model optimizers, --model_config plumbing, README/wiki docs, tests/test_model_registry_pickle.py, and small fixes across gpse/train/* and gpse/convert/genotype_matrix.py.

b99e741 — Release GPSE 0.0.4a

Type: release | **Date:** 2026-08-20

Description: Releases version 0.0.4a, headlined by a --preserve-vcf-snp-ids compatibility mode: genotype matrices and manifests can keep original VCF variant IDs instead of canonical chr_start_end coordinate IDs, with the chosen mode recorded in the feature manifest (feature_id_mode) and enforced/warned about consistently across convert, train, and predict.

Code:

```

-def convert_to_matrix(fileprefix, out_file=None, *, out_format="parquet", geno_encoding="012",
logger=None):
+def convert_to_matrix(
+    fileprefix, out_file=None, *, out_format="parquet", geno_encoding="012",
+    preserve_vcf_snp_ids=False, logger=None,
+):
    ...
+    if os.path.exists(out_file) and os.path.exists(manifest_file):
+        existing_mode = json.load(manifest_handle).get("feature_id_mode", "canonical")
+        requested_mode = "vcf" if preserve_vcf_snp_ids else "canonical"
+        if existing_mode == requested_mode:
+            log.info(f"Matrix file already exists: {out_file}")
+            return out_file
+        log.warning(f"Existing matrix uses SNP ID mode '{existing_mode}'; regenerating.")
    ...

```

```

+     if preserve_vcf_snp_ids:
+         snpid_list = vcf_ids_from_map_file(map_path)
+         feature_id_mode = "vcf"
+     else:
+         snpid_list = canonical_ids_from_map_file(map_path)
+         feature_id_mode = "canonical"
# gpse/utils/snp_ids.py
def vcf_ids_from_map_file(map_file):
    """Read original VCF/PLINK variant IDs from MAP/BIM rows.

    Missing VCF IDs fall back to canonical coordinates so partially annotated
    VCF files still produce valid, unique feature names.
    """
    ...
    feature_ids.append(
        variant_id if variant_id and variant_id != "."
        else canonical_snp_id_from_map(fields[0], fields[3])
    )
# pyproject.toml
-version = "0.0.3a1"
+version = "0.0.4a"

```

Changes: - New `--preserve-vcf-snp-ids` flag on `gpse convert` (`gpse/utils/cli_display.py`), `gpse train` (`gpse/train/workflow.py`), and `gpse predict` (`gpse/predict/cli.py`); threaded through `gpse/convert/processor.py`, `gpse/train/cli.py`, and `gpse/predict/core.py`. - `gpse/utils/snp_ids.py`: new `vcf_ids_from_map_file()` (missing "." IDs fall back to canonical coordinates); `gpse/utils/feature_manifest.py` records `feature_id_mode`. - `gpse/convert/genotype_matrix.py`: cached matrices are regenerated when the requested SNP-ID mode differs from the manifest; conversion and training emit warnings to keep the same mode for prediction. - `gpse/train/predictor.py` gains a `feature_id_mode` parameter passed into manifest writes from `gpse/train/_data_io.py`. - Version bump to 0.0.4a in `pyproject.toml` and `gpse/config/default.yaml`; README documents the compatibility mode; new `tests/test_snp_id_alignment.py` case asserting VCF-ID columns (rs100, rs200) and `feature_id_mode: "vcf"`.